

Session 4 · Deck 4.A2

Optimization in Many Dimensions, and Where Derivatives Come From

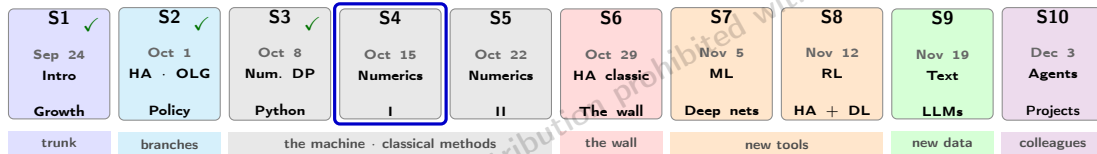
Zhigang Feng

Thursday, October 15, 2026

Build it on paper → solve it on the machine → push it to the frontier with AI

Where We Are

you are here: 4.A2 ▼



Build it on paper → solve it on the machine → push it to the frontier with AI

Block A: 4.A1 root-finding and optimization on an interval · 4.A2 optimization in many dimensions, and where derivatives come from.

What Breaks When the Choice Is a Vector

Every method in 4.A1 **shrank a bracket**; a region of $\mathbb{R}^N$ has no two ends to shrink. What replaces it is Newton. In one dimension an optimum is a root of f' , so 4.A1's Newton applied to f' gives

$$x_{n+1} = x_n - \frac{f'(x_n)}{f''(x_n)}.$$

Read f'/f'' as a distance: a large slope means far from the flat top; a large curvature means the slope is changing fast, so you are close.

In N dimensions the same step solves a local quadratic model,

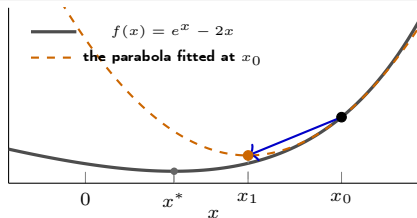
$$f(\theta + \Delta) \approx f(\theta) + g^\top \Delta + \frac{1}{2} \Delta^\top H \Delta \implies \Delta = -H^{-1}g,$$

with gradient $g = \nabla f$ and Hessian $H = \nabla^2 f$.

	1 variable	N variables
curvature	a number, f''	a matrix H , with cross-partials $\partial^2 f / \partial x \partial y$
Newton step	divide by f''	solve $H\Delta = -g$
a minimum needs	$f'' > 0$	H positive definite: curving up in every direction

Cross-partials are interactions: moving x changes the slope in y , so the variables cannot be optimized one at a time. And Newton finds where $g = 0$ — maxima, minima and **saddles** alike; in many dimensions most flat points are saddles.

Newton for a Minimum: Fit a Parabola, Jump to Its Bottom



From $x_0 = 2$: the parabola's bottom is $x_1 = 1.27$, a jump of $-f'/f'' = -5.39/7.39 = -0.73$ toward $x^* = \ln 2$.

Then repeat — the error squares each step, as in 4.A1 (ledger `newton_min_1d`):

n	x_n	$f'(x_n)$	$x_n - x^*$
0	2.000	5.389	1.3
1	1.271	1.563	0.58
2	0.832	0.298	0.14
3	0.702	0.019	9×10^{-3}
5	0.693147	0	9×10^{-10}

A root used the tangent. A minimum uses the **parabola** that matches f at x_n in value, slope and curvature,

$$q(x) = f(x_n) + f'(x_n)(x - x_n) + \frac{1}{2}f''(x_n)(x - x_n)^2,$$

and jumps to its bottom, where $q'(x) = 0$:

$$x_{n+1} = x_n - \frac{f'(x_n)}{f''(x_n)} \quad (\text{slope} \div \text{curvature}).$$

Why the equation to solve is $\nabla f(\theta) = 0$

At the bottom of a bowl no direction goes downhill, so every partial derivative is zero: N equations in N unknowns, the first-order conditions (Euler equations are this). Newton fits a bowl and jumps to its bottom, $H\Delta = -g$ (next frame); the jump lands on the model's flat point whatever its type, and $f'' > 0$, or H positive definite, is what makes it a minimum.

Newton in N Dimensions: One Example, Then the Bill

The running example of this deck: minimize

$$f(x, y) = \frac{1}{2}x^2 + y^2 \text{ from } \theta_0 = (2, 2).$$

1. Gradient: $g_0 = (x, 2y) = (2, 4)$ — **2** numbers.
2. Hessian: $H = \begin{pmatrix} 1 & 0 \\ 0 & 2 \end{pmatrix}$ — **4** numbers.
3. Solve $H\Delta = -g_0$: $\Delta = (-2, -2)$ — a 2×2 system.
4. Step: $\theta_1 = \theta_0 + \Delta = (0, 0)$, where the gradient is zero. One step, because f is exactly quadratic.

At $N = 2$ every piece is trivial. Now count each piece as N grows.

	$N = 10$	100	1,000
gradient entries	10	100	1,000
Hessian entries	100	10,000	1 million
f -evaluations to build H , $1 + N + \frac{N(N+1)}{2}$	66	5,151	501,501
flops to solve $H\Delta = -g$	333	333,333	333 million
memory for H	800 B	80 KB	8 MB

The gradient grows like N ; the Hessian like N^2 to build and store, and the solve like N^3 — **every step**. If each f -evaluation is a model solve, half a million per step ends the method. Hence what follows: quasi-Newton builds no H , and conjugate gradient stores none.

Ledger newton_bill: forward differences use $f(\theta)$, $f(\theta + he_i)$, $f(\theta + he_i + he_j)$ for $i \leq j$; a factorisation costs about $N^3/3$ flops.

Systems of Equations: Newton, and Broyden's Shortcut

$\nabla f(\theta) = 0$ is itself a system of N equations, with Jacobian H ; so is market clearing in N markets. For $F: \mathbb{R}^N \rightarrow \mathbb{R}^N$, Newton becomes

$$J(x_n) \Delta_n = -F(x_n), \quad x_{n+1} = x_n + \Delta_n,$$

with $J_{ij} = \partial F_i / \partial x_j$: solve the *linear* model of F , as one dimension solved the tangent. Two costs per step:

- **building** J : N equations in N unknowns, so N^2 partials; by forward differences, nudge one x_j per evaluation and read off a column — $N + 1$ evaluations of F ;
- **solving** the linear system: $O(N^3)$.

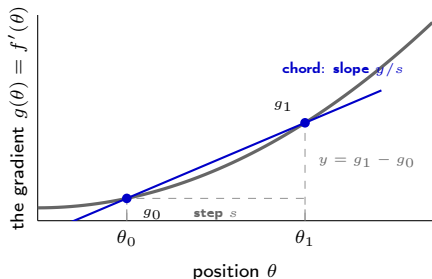
Broyden is the secant method in N dimensions: after each step it corrects J just enough to map the step taken, Δx , to the change observed, ΔF . One Jacobian, at the start.

Broyden (1965). Where the derivatives come from: later in this deck.

$x^2 + y^2 = 4, e^x + y = 1$	iters	Jacobians
Newton	5	5
Broyden	8	1

Both reach a residual at machine precision. Broyden takes about one and a half times the iterations and one fifth of the Jacobians — and when J is the expensive part, that is the trade you want. (Updating J^{-1} directly, by Sherman–Morrison, removes the $O(N^3)$ solve too.)

Quasi-Newton: Learn the Curvature From the Gradients You Already Have



The gradient is a function too, and its slope is the curvature. We never compute that slope — we **read it off two gradients** we already have: the chord's slope y/s is 4.A1's secant, applied to f' .

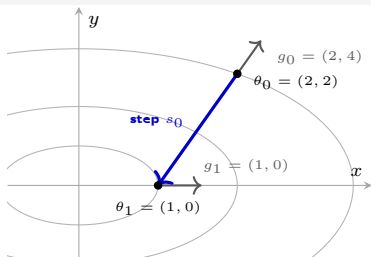
Newton's bill was the curvature H : N^2 numbers, every step. But curvature is only *how fast the slope changes*, and the slope — the gradient — is computed at every point we visit anyway. So read it off two of them.

In one dimension that is the chord's slope, $(g_1 - g_0)/(\theta_1 - \theta_0)$. In N dimensions the same sentence — *the change in the gradient is the curvature times the step* — reads $Bs = y$, with s the **step taken**, y the **change in the gradient**, and B our estimate of H .

A family, and its best-known member

Quasi-Newton is any method that updates B so that $Bs = y$ holds after each step. **BFGS** is the member everyone uses (DFP and SR1 are others); Broyden is the same idea for a system.

BFGS by Example, Step 1: Take a Step and Watch the Gradient Change



Contours of $f = \frac{1}{2}x^2 + y^2$. The first step runs down the gradient; the gradient at the landing point is what the method learns from.

Minimize $f(x, y) = \frac{1}{2}x^2 + y^2$ from $\theta_0 = (2, 2)$, where $g_0 = (2, 4)$. Nothing is known about the curvature yet, so $B_0 = I$ and the first direction is plain steepest descent, $d_0 = -g_0 = (-2, -4)$. Go half-way, $\alpha_0 = 0.5$:

$$\theta_1 = (1, 0), \quad g_1 = (1, 0).$$

What the step revealed:

$$s_0 = \theta_1 - \theta_0 = (-1, -2), \quad y_0 = g_1 - g_0 = (-1, -4).$$

The insight

x moved by 1 and its gradient fell by 1. y moved by 2 and its gradient fell by 4. The slope changes once as fast as the position in x and **twice** as fast in y : the bowl is steeper in y . That is the true $H = \text{diag}(1, 2)$ — which the method has never computed.

Ledger bfgs_worked. $\alpha_0 = 0.5$ is a convenient round number; the exact line-search minimizer is $5/9$.

BFGS by Example, Step 2: Update the Curvature Estimate, Then Repeat

Ask the new estimate to **reproduce the change just seen**:

$B_1 s_0 = y_0$ (the truth does: $H s_0 = (-1, -4) = y_0$). One equation cannot pin down a matrix, so BFGS changes B_0 as *little as possible*:

$$B_1 = B_0 + \underbrace{\frac{y_0 y_0^\top}{y_0^\top s_0}}_{\text{add what was seen}} - \underbrace{\frac{B_0 s_0 s_0^\top B_0}{s_0^\top B_0 s_0}}_{\text{remove the old guess}}$$

Here $y_0^\top s_0 = 9$ and $s_0^\top B_0 s_0 = 5$, so

$$B_1 = \begin{pmatrix} 0.91 & 0.04 \\ 0.04 & 1.98 \end{pmatrix} \quad \text{beside} \quad H = \begin{pmatrix} 1 & 0 \\ 0 & 2 \end{pmatrix}.$$

After **one** step the estimate has the right shape: y twice as curved as x . Then repeat; on a quadratic, B reaches H in N steps.

The loop

- 1: $B \leftarrow I$; $g \leftarrow \nabla f(\theta)$
- 2: **repeat**
- 3: solve $B d = -g$ ▷ direction
- 4: $\alpha \leftarrow$ line search; $s \leftarrow \alpha d$; $\theta \leftarrow \theta + s$
- 5: $y \leftarrow \nabla f(\theta) - g$; $g \leftarrow g + y$
- 6: $B \leftarrow B + \frac{y y^\top}{y^\top s} - \frac{B s s^\top B}{s^\top B s}$
- 7: **until** $\|g\| < \varepsilon$

What it buys

No second derivatives, ever. Updating B^{-1} directly (Sherman–Morrison) makes a step $O(N^2)$ instead of $O(N^3)$; L-BFGS keeps only the last few (s, y) pairs and is $O(N)$.

The Step Size: What a Line Search Is For

Step 1 took $\alpha_0 = 0.5$ because the arithmetic was clean.

In general the update gives a *direction* d ; how far to move along it is a separate, one-dimensional problem:

$$\varphi(\alpha) = f(\theta + \alpha d), \quad \alpha > 0.$$

Solving it exactly is wasteful — the direction will change next step anyway. What is needed is a step that makes **enough** progress:

- **Armijo**: f falls by a fixed fraction of what the slope promised.
- **Curvature**: the slope has flattened enough that we are not stopping too early.

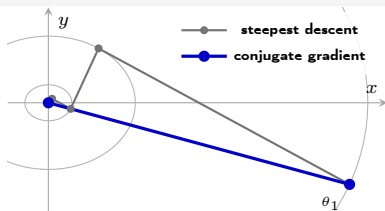
Together these are the **Wolfe conditions**. Backtracking from $\alpha = 1$ enforces the first; BFGS codes also check the second.

This is 4.A1, nested

The inner problem is exactly the scalar maximization of the previous deck — which is why golden section and Brent turn up inside multivariate optimizers.

The curvature condition is not optional for BFGS: it is what guarantees $y^\top s > 0$, and hence that B stays positive definite. Skip it and the update can produce a direction that is not even downhill.

Conjugate Gradient: Do Not Undo Your Last Step



Contours of $\frac{1}{2}x^2 + y^2$ near the minimum. Both methods take the same first step, to θ_1 . Then steepest descent zig-zags (**16** steps to 10^{-8}); conjugate gradient goes straight there: **two** steps in all.

Why steepest descent zig-zags. An exact line search stops where the new gradient is perpendicular to the direction just used, so the next step crosses back over it: part of the last step is undone.

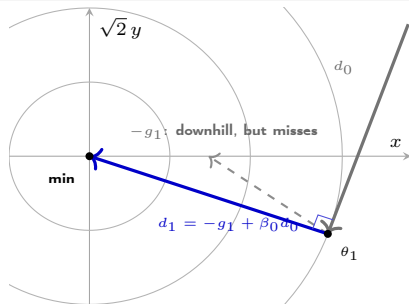
The fix. Bend each new direction just enough that moving along it *leaves the previous direction's progress intact*: “perpendicular” in a round bowl, “perpendicular once the bowl is stretched round” in an elliptical one:

Conjugate directions

$d_i^T H d_j = 0$ for $i \neq j$: perpendicular, measured with the curvature. On the page d_0 and d_1 are not; they are once y is stretched by $\sqrt{2}$ — the next frame builds d_1 in that picture.

The pay-off. Each direction is finished for good: for a quadratic in N dimensions, N steps hit the minimum **exactly** — Newton's answer, with no H built or stored.

The Second Direction: Bend It Until It Points at the Centre



The bowl with y stretched by $\sqrt{2}$: circles. The line search along d_0 ended at θ_1 ; the direction that keeps d_0 finished is the **perpendicular** one, and in a round bowl it passes through the centre. $-g_1$ is downhill but not perpendicular here.

Bend $-g_1$ by a dose of d_0 . A step along $-g_1$ changes the slope along d_0 at the rate $d_0^\top H(-g_1) = -1.78$: it re-opens what step 1 closed. A step along d_0 changes that slope at the rate $d_0^\top H d_0 = 36$. Add just enough d_0 to cancel:

$$d_1 = -g_1 + \beta_0 d_0, \quad \beta_0 = \frac{1.78}{36} = 0.049,$$

so the slope along d_0 stays zero as you move: $d_0^\top H d_1 = 0$. With exact line searches on a quadratic this β_0 equals $g_1^\top g_1 / g_0^\top g_0$, the loop's formula — no H needed.

Compared with BFGS: same ingredients, no matrix

At θ_1 both methods hold the same facts: d_0 , g_0 , g_1 . BFGS packs them into a matrix B_1 (from s_0 , y_0) and solves $B_1 d_1 = -g_1$; CG packs them into **one number**, β_0 . With the exact line search both give $d_1 = (-0.99, 0.25)$: the same second step, two book-keepings. What differs is storage, N^2 numbers against two vectors (Nazareth 1979; ledger `cg_vs_bfgs`).

CG by Example: Two Steps to the Minimum

The running example again, from $\theta_0 = (2, 2)$ with $g_0 = (2, 4)$.

1. **Steepest descent first.** $d_0 = -g_0 = (-2, -4)$; the exact line search gives $\alpha_0 = 0.56$, so $\theta_1 = (0.89, -0.22)$ and $g_1 = (0.89, -0.44)$.
2. **A bent direction.** Steepest descent at θ_1 , plus the dose $\beta_0 = g_1^\top g_1 / g_0^\top g_0 = 0.05$ of the old direction that cancels the interference (previous frame):

$$d_1 = -g_1 + \beta_0 d_0 = (-0.99, 0.25).$$

Check: $d_0^\top H d_1 = 0$ — it does not interfere with step 1. In the picture it points straight at the minimum.

3. **Step along it.** $\alpha_1 = 0.9$: $\theta_2 = (0, 0)$. The minimum, in $N = 2$ steps.

The loop

- 1: $g \leftarrow \nabla f(\theta)$; $d \leftarrow -g$
- 2: **repeat**
- 3: $\alpha \leftarrow$ line search along d ; $\theta \leftarrow \theta + \alpha d$
- 4: $g' \leftarrow \nabla f(\theta)$; $\beta \leftarrow g'^\top g' / g^\top g$
- 5: $d \leftarrow -g' + \beta d$; $g \leftarrow g'$
- 6: **until** $\|g\| < \varepsilon$

Newton, in N cheap steps

The N conjugate directions form a basis; minimizing along each in turn solves $H\Delta = -g$ without ever forming H . Memory: a few vectors, $O(N)$. On a non-quadratic f the line search is a real one.

The Memory Wall

The reason CG exists is not elegance. It is that B does not fit.

N	Newton / BFGS	L-BFGS	CG
10^2	10^4	2,000	300
10^4	10^8	2×10^5	3×10^4
10^6	10^{12}	2×10^7	3×10^6

At $N = 10^6$ the dense approximation is 10^{12} doubles — **8 terabytes**. L-BFGS needs 160 MB and CG needs 24.

L-BFGS

Keep the last $m \approx 10$ pairs (s, y) and rebuild the action of B^{-1} on demand. $O(mN)$ memory, and almost all of BFGS's behaviour.

$N = 10^6$ is not hypothetical. It is the parameter count of a small neural network — which is why **no one trains one with Newton's method**, and why Session 7's optimizer is a first-order one.

When the Objective Is a Sum Over Data

A different large problem: an average over M observations,

$$L(\theta) = \frac{1}{M} \sum_{i=1}^M \ell(\theta; z_i)$$

— an empirical risk, an SMM objective, a panel likelihood.

The challenge: one gradient, $\nabla L = \frac{1}{M} \sum_i \nabla \ell(\theta; z_i)$, **touches every observation**. Each BFGS or CG iteration is a full pass over the data — M per-observation gradients (for SMM, M simulated paths) to choose one direction; with $M = 10^6$, a million per iteration.

Stochastic gradient descent averages over a random *mini-batch* $\mathcal{B}_n$ of $B \ll M$ observations instead:

$$\theta_{n+1} = \theta_n - \alpha_n \frac{1}{B} \sum_{i \in \mathcal{B}_n} \nabla \ell(\theta_n; z_i).$$

Why most of a full pass is wasted

When observations agree with each other, a hundred of them point almost the same way as a million. The noise in a mini-batch gradient falls only like $1/\sqrt{B}$: going from $B = 100$ to 10^6 buys a hundred times the precision for ten thousand times the work.

Why it is not madness

The mini-batch gradient is **unbiased**: averaged over the draw, it is the full gradient. Each step is cheap and right on average — and wrong on any single draw. The next frame shows what that looks like.

SGD on Our Example: The Noise, and Why the Step Must Shrink

The previous frame's claims on the running function: treat $\ell_1 = \frac{1}{2}x^2$ and $\ell_2 = y^2$ as two observations ($M = 2$) and draw one per step, $\alpha = 0.5$. At $(2, 2)$ a draw gives $(2, 0)$ or $(0, 4)$: their average, $(1, 2)$, is the full gradient of L — yet each draw moves only one coordinate.

θ_n	drawn	θ_{n+1}
$(2, 2)$	ℓ_2	$(2, 0)$
$(2, 0)$	ℓ_1	$(1, 0)$
$(1, 0)$	ℓ_1	$(0.5, 0)$
$(0.5, 0)$	ℓ_2	$(0.5, 0)$
$(0.5, 0)$	ℓ_1	$(0.25, 0)$

Row four makes no progress at all — the draw carried no information about x . That is the noise, and it is the price of the cheap step.

Here $f = \ell_1 + \ell_2$ is M times the average L , with the same minimiser, so $\nabla f(2, 2) = [2, 4]$ as in the BFGS and CG examples.

Why α must be small, and shrink

Each sampled gradient is the true gradient plus noise. Noise of unit size in each of N coordinates has norm $\sqrt{N}$: 1, 100 and 1,000 at $N = 1, 10^4$ and 10^6 — so a step safe in one dimension is a thousand times too large in a million. And to settle at the optimum rather than bounce around it, the steps must shrink over time: $\sum \alpha_n = \infty$ (you can still reach anywhere) and $\sum \alpha_n^2 < \infty$ (the noise adds up to something finite) — Robbins–Monro.

When You Have No Derivatives at All

Nelder–Mead keeps a simplex of $N + 1$ points and moves it by reflecting the worst vertex through the centroid, then expanding, contracting or shrinking. No derivatives, no model of f , and no convergence theory worth the name — but it is robust to noise and kinks.

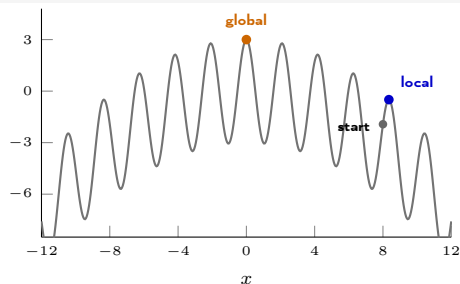
Rosenbrock from $(-1.2, 1)$	evals	$ \theta - \theta^* $
BFGS	117	9×10^{-6}
Nelder–Mead	249	5×10^{-11}
Powell	607	2×10^{-13}

Read that carefully

BFGS is fastest by a factor of two, and *least* accurate by five orders of magnitude — run at its default gradient test (10^{-5}) while Nelder–Mead was given 10^{-10} . Each stops when *its* own test passes.

“Fastest” and “most accurate” are different questions. Decide which one you are asking **before** you pick the method, and report the tolerance you used.

Simulated Annealing: Accept a Worse Point, On Purpose



$f(x) = -x^2/20 + 3 \cos 3x$. From $x_0 = 8$ a local method climbs the nearest hill and stops at $x = 8.35$, $f = -0.50$. The top, $f = 3.00$ at $x = 0$, lies several valleys away — and to get there you must first go *down*.

From $x_0 = 8$ annealing ends at $x = 0.00$, $f = 3.00$: the global maximum, for 20,000 evaluations.

Every method so far only goes up, so it stops on its starting hill. Annealing walks down on purpose:

1. propose a random move $x^* = x + \text{noise}$;
2. if $f(x^*) \geq f(x)$, accept — uphill is always taken;
3. if worse by a drop $\Delta = f(x) - f(x^*) > 0$, accept with probability $e^{-\Delta/T}$: draw u uniform on $[0, 1]$ and accept if $u < e^{-\Delta/T}$;

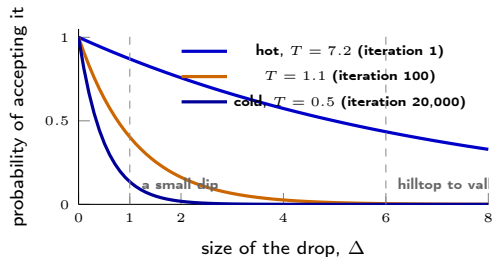
then lower the temperature T a little, and repeat.

Why “temperature”

Heat a metal and its atoms move freely; cool it slowly and they settle into the lowest-energy crystal. Quench it and defects freeze in. Annealing copies the schedule: hot, the walk crosses valleys; cold, it only climbs. The next frame says how often it goes down.

Ledger simulated_annealing, annealing_picture: $T_n = 5 / \ln(n + 1)$; cooling this slowly reaches the global optimum in probability.

Why Accept a Worse Point, and How Often



accept a drop of	$T = 7.2$	$T = 1.1$	$T = 0.5$
$\Delta = 1$	0.87	0.40	0.14
$\Delta = 6$	0.44	0.004	0.000007

(i) **The size of the drop.** The probability falls *exponentially* in Δ : a dip of 1 is almost always accepted, a drop of 6 — hilltop to valley floor here — much less. Shallow valleys are crossed freely; deep ones rarely.

(ii) **The temperature sets what counts as small.** Hot, nearly every move is accepted: a random walk that **explores**. Cold, a drop of 6 is accepted seven times in a million: hill-climbing that **exploits** the hill it is on. Slow cooling moves from one to the other.

(iii) **Why a coin, not a cut-off.** “Accept any drop below $\bar{\Delta}$ ” would trap the walk in every valley deeper than $\bar{\Delta}$. A probability leaves every crossing possible, at a rate T controls.

At a fixed T the walk's long-run distribution is $\propto e^{f(x)/T}$: flat when hot, piled onto the maximum as $T \rightarrow 0$ — so the same rule at a fixed T *samples* (next frame).

One Accept–Reject Rule, Two Very Different Uses

The rule of the last two frames is not really an optimization idea. It is the **Metropolis–Hastings** rule, and the only thing that changes between its two uses is what you do with the temperature.

	annealing	Metropolis–Hastings
accept w.p.	$\min\{1, e^{-\Delta/T_n}\}$	$\min\{1, \frac{\pi(\theta^*)}{\pi(\theta_n)}\}$ (symmetric proposal)
temperature	driven to 0	held at 1
you want	the argmax	the distribution
output	one point	a whole chain

Why an economist cares

Hold the temperature at one and the chain's stationary distribution *is* the posterior. That is how a structural model is estimated when the likelihood can be evaluated but not maximised reliably.

We use the optimization half today. The sampling half returns in **Session 9**, where calibration and estimation are the bridge from models to data — and where the object being sampled is a posterior over parameters rather than a maximum.

When Constraints Bind

Almost every economic optimization is constrained:

$c > 0$, $n \in [0, 1]$, $a' \geq -\underline{a}$. At an optimum the Karush–Kuhn–Tucker conditions hold: $\nabla_x \mathcal{L} = 0$, $\mu \geq 0$, $h(x) \geq 0$, and

$$\mu h(x) = 0.$$

It is the last one — **complementary slackness** — that is awkward numerically: it is not an equation you can hand to a solver.

The Fischer–Burmeister function turns it into one:

$$\phi(a, b) = a + b - \sqrt{a^2 + b^2},$$

$$\phi(a, b) = 0 \iff a, b \geq 0, ab = 0.$$

Where you met this

2.A1 used exactly this ϕ to write the borrowing-constrained household's optimality conditions as a square system — and pointed here for how to solve it.

With ϕ in hand the KKT system becomes $F(x, \mu) = 0$, and this deck's Newton for systems applies directly: a Jacobian, a solve, a line search.

ϕ is not differentiable at the origin — exactly where the constraint is marginally binding — so solvers smooth it and drive the smoothing to zero.

Choosing, and a Warning About Worked Examples

Method	Use it when
Newton	N small, H cheap and reliable
BFGS	the default for smooth f up to a few hundred parameters
L-BFGS	thousands of parameters; only $O(N)$ memory
Conjugate gradient	N in the millions; gradients only, a few vectors of memory
Nelder–Mead	f noisy, kinked, or a black box
Simulated annealing	many local optima, no derivatives, and time to spare

Re-run the arithmetic

Every worked example in this deck was recomputed by machine. In the conjugate-gradient example, change one digit of the second direction and the next step no longer lands on the minimum.

Worked examples are the most useful thing in a numerical-methods course and the easiest place for an error to survive, because nobody re-runs arithmetic that is already typeset. **Re-run it.**

Everything So Far Assumed You Had Derivatives

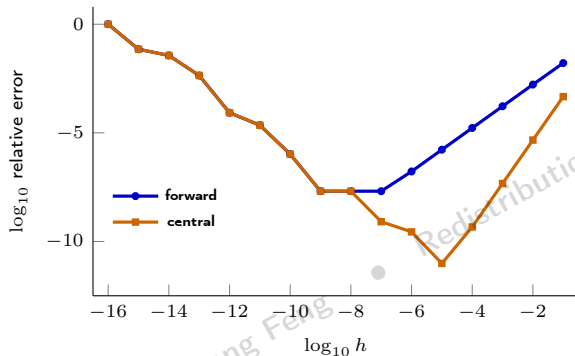
Newton needs a gradient and a Hessian. BFGS and CG need a gradient. A Jacobian needs N^2 partial derivatives. Every method in the last two decks has quietly assumed that ∇f arrives from somewhere.

For a model you wrote down, it does — you can differentiate by hand, once, and check it. For a model with a hundred equations, or a solver nested inside another solver, hand-differentiation stops being realistic. And the obvious substitute, a finite difference, has a failure mode we have already measured.

The rest of this deck

Where derivatives come from: forward and central differences; the **two errors** every finite difference carries, derived one at a time, and the step size that balances them; then automatic differentiation, which carries neither.

Two Formulas, One Derivative



The derivative of k^α at $k = 2$, computed two ways as the step h shrinks:

$$\frac{f(x+h) - f(x)}{h} \quad \text{and} \quad \frac{f(x+h) - f(x-h)}{2h}.$$

	best $\log_{10}$ error
forward difference	-7.7
central difference	-11.0

3.B3 showed you the first curve and called it a floor. The next three frames explain the floor — one error at a time — and show that it moves with the method, and vanishes under automatic differentiation.

Error 1, Truncation: What the Formula Leaves Out

Taylor's theorem **with its remainder** is exact, not approximate: for some ξ between x and $x + h$,

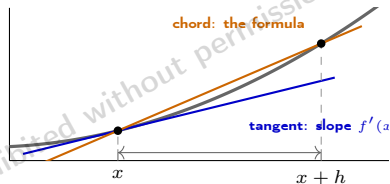
$$f(x + h) = f(x) + h f'(x) + \frac{h^2}{2} f''(\xi).$$

Subtract $f(x)$ and divide by h :

$$\frac{f(x + h) - f(x)}{h} - f'(x) = \frac{h}{2} f''(\xi).$$

So the forward difference misses $f'(x)$ by $\frac{h}{2}|f''|$: the h^2 term of the expansion, divided by h . It is **proportional to h** — **first order**. Halve the step, halve the error.

Truncation **falls** with h : on its own it says take the smallest step you can. The computer adds a second error that says the opposite (next frame).



The formula measures the chord; we want the tangent. The curve bends between x and $x + h$ — that bend is f'' , and the gap between the slopes is $\frac{h}{2} f''$.

Central difference. Expand $f(x - h)$ too; its f'' term has the *same* sign and cancels in the subtraction:

$$\frac{f(x + h) - f(x - h)}{2h} - f'(x) = \frac{h^2}{6} f'''(\xi).$$

Second order: halve the step, *quarter* the error.

Error 2, Round-Off: What the Computer Adds

The computer stores not $f(x)$ but the nearest point of 3.A1's float grid,

$$\widehat{f(x)} = f(x)(1 + \delta), \quad |\delta| \leq \varepsilon \approx 2.2 \times 10^{-16},$$

so each stored value is off by up to $\varepsilon|f|$. Follow that through the formula:

1. two stored values, each off by up to $\varepsilon|f|$;
2. their **difference** is off by up to $2\varepsilon|f|$ (the errors can pull opposite ways);
3. **divide by** h : the error is divided too, $2\varepsilon|f|/h$ — **large** when h is tiny.

Add the two errors:

$$E(h) \leq \underbrace{\frac{h}{2}|f''|}_{\text{truncation}} + \underbrace{\frac{2\varepsilon|f|}{h}}_{\text{round-off}} \quad \left(\text{central: } \frac{h^2}{6}|f'''| + \frac{2\varepsilon|f|}{h} \right).$$

$$f = k^\alpha \text{ at } k = 2$$

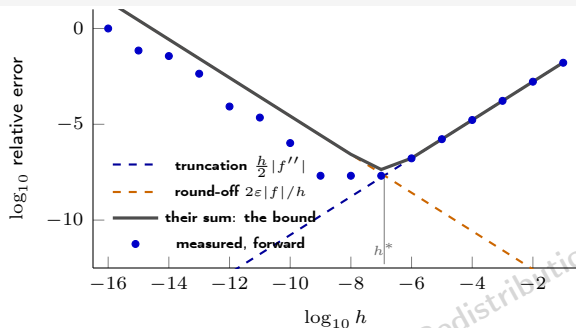
	$h = 10^{-7}$	$h = 10^{-8}$
truncation $\frac{h}{2} f'' $	3.5×10^{-9}	3.5×10^{-10}
round-off $2\varepsilon f /h$	5.6×10^{-9}	5.6×10^{-8}
sum	9.1×10^{-9}	5.6×10^{-8}

Two errors, opposite directions

Shrinking h from 10^{-7} to 10^{-8} cuts truncation tenfold and multiplies round-off tenfold. Neither error can be driven to zero without inflating the other — so there is a best h , and the next frame finds it.

Ledger fd_error_terms: $|f| = 1.26$, $|f''| = 0.069$. The bound is a worst case; the measured error sits a little below it.

Balance the Two: Where the Best Step Comes From



The two lines cross, and the sum is smallest near the crossing. Setting $E'(h) = 0$:

$$\frac{1}{2}|f''| = \frac{2\varepsilon|f|}{h^2} \implies h^* = 2\sqrt{\frac{\varepsilon|f|}{|f''|}}.$$

Here $h^* = 1.3 \times 10^{-7}$, where the bound is $10^{-7.4}$; the measurement bottoms out at $10^{-7.7}$, at $h = 10^{-7}$. With $|f|$, $|f''|$ of order one, $h^* \approx \sqrt{\varepsilon}$ and the error $\approx \sqrt{\varepsilon}$: **half the digits, gone**. Central: $\frac{h^2}{6}|f'''|$ against the same round-off gives $h^* \sim \varepsilon^{1/3}$.

	h^*	error
forward, predicted	1.3×10^{-7}	$10^{-7.4}$
forward, measured	10^{-7}	$10^{-7.7}$
central, predicted	3×10^{-5}	$10^{-9.9}$
central, measured	10^{-5}	$10^{-11.0}$

Ledger fd_error_terms, differentiation. A more accurate formula tolerates a larger step and so loses fewer digits to round-off.

Automatic Differentiation: Neither Symbolic Nor Numerical

AD applies the chain rule to the program, one elementary operation at a time. No step size, no truncation, no cancellation — the result is exact to machine precision.

- **Forward mode** *pushes*: “if I nudge input j , how does each intermediate move?”, carried alongside each value. Cost $\propto$ number of *inputs*.
- **Reverse mode** *pulls*: records the operations, then walks the tape backwards asking “how much does the output care about each intermediate?”. Cost $\propto$ number of *outputs*.

A gradient has one output and many inputs, so reverse mode gives the whole thing for a small constant multiple (typically 3–5) of **one** function evaluation, whatever the number of inputs. That is why it is what trains neural networks.

The mechanics are 3.B3's autograd tape.

Which mode for a Jacobian?

N inputs and N outputs: forward mode costs N sweeps, reverse mode costs N sweeps. Neither wins, and structure — sparsity — is what actually helps.

AD differentiates **the program you wrote**, not the function you meant. Branches, clipping and `max` all have derivatives, and at a kink the one you get is whichever branch executed.

Choosing a Derivative

Method	Accuracy	Use it when
Forward difference	$\sqrt{\epsilon}$, about 8 digits	f is a black box and cheap the default for a black box you control the source; gradients of many inputs the expression is small, and you will check it
Central difference	$\epsilon^{2/3}$, about 11 digits	
Automatic differentiation	exact	
Symbolic / by hand	exact	

The step size is not a free parameter to be tuned. It has a right answer, $\sqrt{\epsilon_{\text{mach}}}$ or $\epsilon_{\text{mach}}^{1/3}$, and choosing it by trial and error is how people end up on the wrong side of the U-curve.

Takeaways

- No bracket in $\mathbb{R}^N$: Newton's step $\Delta = -H^{-1}g$ replaces it, and the **Hessian** is what does not scale — a million entries at $N = 1,000$, every step.
- **Quasi-Newton** reads the curvature off two gradients, $Bs = y$: Broyden for systems, BFGS for optimization — one step, and B knew y was twice as curved as x . The step length is **4.A1, nested**.
- **Conjugate gradient** bends each direction so it never undoes the last: two steps against sixteen for steepest descent, no matrix stored. CG and L-BFGS survive when H stops fitting (10^{12} entries at $N = 10^6$).
- **SGD**: a cheap, unbiased, noisy gradient from a mini-batch, with a shrinking step (Session 7).
- **Fast and accurate are different questions**: on Rosenbrock, BFGS used half the evaluations of Nelder–Mead and finished five orders of magnitude further away.
- **Annealing** accepts a drop Δ with probability $e^{-\Delta/T}$: small drops often, deep ones only while hot; at a fixed temperature the same rule samples a posterior (Session 9). Constraints become equations through Fischer–Burmeister.
- A finite difference carries **two errors**, truncation $\frac{h}{2}|f''|$ falling with h and round-off $2\epsilon|f|/h$ rising; balancing them puts the best step at $\sqrt{\epsilon}$ (forward) or $\epsilon^{1/3}$ (central), where the measurement lands.
- **Automatic differentiation is exact**; in reverse mode a gradient costs about one evaluation, which makes Session 7 possible. It differentiates the program: at a kink you get the branch that ran.

Bridge: After the Break, Representing Functions

Block A has been about **operating** on a function you already have: finding where it is zero, where it is largest, how steep it is. Every one of those methods asked to evaluate f at a point of its choosing — and in a dynamic program, $\hat{v}$ is only known at N grid points.

So the questions change. How do you represent a function you only know at finitely many places? Which representation keeps the properties the algorithms of Block A depend on? How do you turn the expectation in the Bellman equation into a sum — and where does the transition matrix Π , which we have simply written down twice now, actually come from?

After the break (Block B)

4.B1 Approximation and interpolation.

4.B2 Expectations: quadrature, and discretizing the shock — Tauchen and Rouwenhorst, and which one you should actually use.

4.B3 Fast and accurate: how Howard, monotonicity, concavity and the endogenous grid method each work.

4.B4 The worked example: 3.A4's solver, rebuilt with all of it.